Algorithm ?? vs. BFGS on the MGH Problems

1 The degeneracy at $k = 0$

Algorithm ?? minimizes $f(x) = \frac{1}{2}\|F(x)\|^2$, where $F = (f_1, \dots, f_m)^\top$ is the residual vector. At external iteration k it keeps one symmetric matrix H_i^k per residual component f_i, approximating $\nabla^2 f_i(x^k)$, and solves

$$M_k(s) \; = \; \sigma\|s\|^2 + \tfrac{1}{2}\sum_i q_i(s)^2, \qquad q_i(s) = f_i(x^k) + \nabla f_i(x^k)^\top s + \tfrac{1}{2} s^\top H_i^k s, \tag{1}$$

accepting or rejecting $x^k + s$ by the ratio between actual and predicted reduction, and growing/shrinking σ accordingly. At $k = 0$, every $H_i^0 = 0$ (no secant pair has been observed yet), so (1) collapses to

$$M_0(s) = \sigma\|s\|^2 + \tfrac{1}{2}\sum_i \big(f_i(x^0) + \nabla f_i(x^0)^\top s\big)^2, \tag{2}$$

a single damped Gauss–Newton subproblem. Two things are worth noting about this particular instance of the subproblem. Unlike every subsequent iteration, $\sigma = \sigma_0 = 1$ here is a fixed constant, not yet informed by any accept/reject test on *this* problem, so how close (2)'s solution already sits to the scaled steepest-descent direction it approaches as $\sigma \to \infty$ (where the minimizer of (2) is asymptotically parallel to $-\nabla f(x^0)$) is essentially a guess about the local curvature scale of f at x^0 that the method has had no chance to calibrate yet. Moreover, there is, by construction, no curvature information at all to justify solving a quartic (rather than a quadratic) model here. The quartic term $\frac{1}{2} s^\top H_i^k s$ is identically zero for every i. So the very first step is the one iteration where the method's own machinery (quasi-Newton curvature, calibrated regularization) has nothing to work with.

1.1 The closed form of M_0

Rather than appealing to a different algorithm, the cleanest justification comes directly out of (2) itself, which is an unconstrained, strictly convex quadratic in s and can be solved in closed form. Write $J := F'(x^0)$ for the Jacobian of F at x^0 (the matrix whose i-th row is $\nabla f_i(x^0)^\top$), so that $g := \nabla f(x^0) = J^\top F(x^0)$, and $A := J^\top J$ (symmetric positive semidefinite). Then

$$\nabla_s M_0(s) = (2\sigma I + A)\, s + g = 0 \quad \Longrightarrow \quad s(\sigma) = -(A + 2\sigma I)^{-1} g.$$

Lemma 1 (Descent for every $\sigma > 0$)**.** *For every $\sigma > 0$ and every $g \neq 0$, $s(\sigma)$ is a descent direction for f at x^0, satisfying $g^\top s(\sigma) < 0$.*

Proof. Since $A + 2\sigma I \succ 0$ for $\sigma > 0$, its inverse is symmetric positive definite, hence

$$g^\top s(\sigma) = -g^\top (A + 2\sigma I)^{-1} g < 0 \qquad \text{whenever } g \neq 0. \qquad \square$$

So $s(\sigma)$ is a genuine descent direction for f *for every* $\sigma > 0$, not just asymptotically. The σ-loop never has a "wrong-direction" problem at $k = 0$; what it can have is a badly-*sized* step, which is what the accept/reject test and the growth of σ are actually correcting for.

Proposition 1 (Alignment with $-g$ as $\sigma \to \infty$)**.** *As $\sigma \to \infty$,*

$$s(\sigma) = -\frac{1}{2\sigma}\, g + o\!\left(\frac{1}{\sigma}\right) \qquad \Longrightarrow \qquad \frac{s(\sigma)}{\|s(\sigma)\|} \longrightarrow -\frac{g}{\|g\|}. \tag{3}$$

Proof. Factor 2σ out of the inverse:

$$s(\sigma) = -(A + 2\sigma I)^{-1} g = -\frac{1}{2\sigma}\left(I + \frac{A}{2\sigma}\right)^{-1} g.$$

As $\sigma \to \infty$, $A/(2\sigma) \to 0$, and matrix inversion is continuous at the identity, so $\big(I + A/(2\sigma)\big)^{-1} \to I$, which gives the stated expansion. □

In words, for σ large enough, the solution of the degenerate $k = 0$ subproblem (2) already is, to leading order, a steepest-descent step of length $\approx 1/2\sigma$. The two are asymptotically the same computation. This is a direct consequence of (2)'s own closed form, entirely internal to Algorithm **??**.

In practice, the rejection ladder $\sigma_0,\ \sigma_{j+1} = \gamma\sigma_j$ ($\gamma > 1$ the growth factor) tries $s(\sigma_0), s(\sigma_1), \ldots$ until one is accepted. By Proposition 1, once $\sigma_j \gg \|A\|/2$ the successive trials are already essentially parallel to $-g$ and shrink by the fixed factor $1/\gamma$, so once it reaches that scale, the ladder is already behaving like a backtracking line search along $-g$. This is also a reason not to let σ_0 run away unchecked. Because σ is only shrunk, never reset, between outer iterations, an unusually large σ_0, forced whenever the true first step happens to be small, persists into σ_1 and can over-dampen the next iteration's subproblem as well.

2 Computational tests

In this section we evaluate the performance of Algorithm **??** and compare it against a classical sum-of-squares approach. We run two types of tests. In the first, we consider the MGH (Moré, Garbow, and Hillstrom) sum-of-squares test problems [8], a benchmark of 34 classical problems; this part is important to us because it helps us settle on a good solver and Hessian approximation. In the second part, we solve a hydraulic problem in which we need to estimate a coefficient of a PDE called the Saint-Venant equation.

2.1 Algorithm ?? for MGH problems

Algorithm **??** minimizes $f(x) = \frac{1}{2}\|F(x)\|_2^2$ (Section 3, 1) by minimizing, at each iteration, a local quartic model of the residuals in place of a line search. Following Algorithm **??** (Eq. (21)), the step s at iteration k solves

$$M_k(s) = \sigma\|s\|_2^2 + \frac{1}{2}\sum_{i=1}^{m} q_i(s)^2, \qquad q_i(s) = f_i(x_k) + \nabla f_i(x_k)^\top s + \frac{1}{2} s^\top H_i s,$$

where f_i and ∇f_i are the i-th residual and its gradient at x_k (Section 3), H_i is the current quasi-Newton approximation to its Hessian. Four further variants are compared in Table 1, and $\sigma\|s\|_2^2$ is the Levenberg-Marquardt-style regularization of Algorithm **??** that damps the step in place of a line search. The candidate $x_k + s$ is checked for feasibility as in Step 3 (Eq. **??**); acceptance then uses the ratio ρ_k between the actual and the model-predicted reduction in f, a refinement of Algorithm **??**'s simpler decrease test (Eq. (25)), accepting whenever $\rho_k \geq \eta_1$ for a fixed threshold η_1.

The implementation tested below uses the regularization update of Eq. (**??**) with the following parameters. The first iteration starts at $\sigma = 1$. Each accepted step divides σ by a fixed factor $\sigma_{grow} = 10$, and each rejected step, as in Eq. (30), multiplies σ by the same factor $\sigma_{grow} = 10$ and re-solves the subproblem at the same point, until a step is accepted; between outer iterations, σ always carries over from the previous one rather than being reset. The floor $\sigma_{\min} = 10^{-8}$ and the ceiling $\sigma_{\max} = 10^{12}$ are used purely for numerical stability, preventing σ from underflowing or overflowing over the course of a run.

Every run reported below stops under one of three point-wise criteria, evaluated with the same default tolerances for Algorithm **??** and for Optim.jl's `BFGS` [7] (kept equal so the comparison is fair):

- **Gradient**: the gradient norm reaches $\|\nabla f\| \leq 10^{-8}$. This is a genuine first-order stationary point.

- **Step**: the last step has (numerically) zero length. The solver stalls and stops making progress.
- **Function**: the objective value does not change between two consecutive iterations. The iterate is stuck on a plateau.
- **Stalled**: Algorithm **??**'s regularization parameter exceeded its ceiling.

Table 1 compares the six Algorithm **??** Hessian-update models, each run independently on the 34 MGH least-squares problems [8]. Its "Converged (gradient)" column counts how many of the 34 problems stopped under the gradient criterion; the remaining columns are mean ± standard deviation over the 34 problems. Each model keeps its own set of matrices $H_i \approx \nabla^2 f_i(x_k)$, $i = 1, \dots, m$, updated at every accepted step from $s = x_{k+1} - x_k$ and $y_i = \nabla f_i(x_{k+1}) - \nabla f_i(x_k)$:

- **BFGS**: the classical rank-two secant update (Eq. (19)),

$$H_i \leftarrow H_i - \frac{H_i s\, s^T H_i}{s^T H_i s} + \frac{y_i y_i^T}{s^T y_i}.$$

- **SR1**: the symmetric rank-one update (Eq. (20)),

$$H_i \leftarrow H_i + \frac{(y_i - H_i s)(y_i - H_i s)^T}{(y_i - H_i s)^T s}.$$

- **DW** (Dennis-Wolkowicz [3], Theorem 4.5(iii)): a Broyden-class update that adds a third, symmetrizing rank-one term to the BFGS pair,

$$H_i \leftarrow H_i - \frac{H_i s\, s^T H_i}{s^T H_i s} + \frac{y_i y_i^T}{s^T y_i} + (s^T y_i)\, w_i w_i^T, \qquad w_i = \frac{y_i}{s^T y_i} - \frac{H_i s}{s^T H_i s}.$$

- **PSB** (Powell-Symmetric-Broyden [9]): a rank-two update that does not preserve positive definiteness (like SR1),

$$H_i \leftarrow H_i + \frac{(y_i - H_i s)s^T + s(y_i - H_i s)^T}{s^T s} - \frac{s^T (y_i - H_i s)}{(s^T s)^2}\, s s^T.$$

Table 1: Comparison of the Algorithm **??** Hessian-update models on the 34 MGH problems.

Model	Converged (gradient)	Iterations	Func. evals	Grad. evals	Time (s)
BFGS	29/34	56.88 ± 188.25	99.97 ± 351.85	57.88 ± 188.25	2.071 ± 6.121
SR1	32/34	23.12 ± 54.84	44.32 ± 112.92	24.12 ± 54.84	0.934 ± 2.992
DW	29/34	56.12 ± 181.91	103.24 ± 351.45	57.12 ± 181.91	1.865 ± 5.942
PSB	31/34	15.68 ± 22.53	31.38 ± 53.44	16.68 ± 22.53	0.424 ± 0.919

PSB and SR1 are the two best models. Together they solve the most problems under the gradient criterion, and PSB in particular does so with by far the fewest iterations, evaluations, and execution time on average (Table 1). SR1 nonetheless certifies gradient convergence on one more problem than PSB (32/34 against 31/34). To see exactly where they disagree, Table 2 isolates the three problems on which at least one of the two methods does not stop under the gradient criterion.

Table 2: Algorithm ?? (PSB) vs. Algorithm ?? (SR1): per-problem comparison on the MGH problems.

Problem	Method	f	$\|\nabla f\|$	Func. evals	Grad. evals	Criterion
MGH10	Algorithm ?? (PSB)	4.40×10^{1}	5.75×10^{-5}	299	127	step
MGH10	Algorithm ?? (SR1)	4.40×10^{1}	5.50×10^{-2}	297	125	step
MGH16	Algorithm ?? (PSB)	4.29×10^{4}	9.26×10^{-5}	98	36	function
MGH16	Algorithm ?? (SR1)	4.29×10^{4}	6.26×10^{-5}	44	9	function
MGH31	Algorithm ?? (PSB)	1.53	2.53×10^{-8}	62	21	function
MGH31	Algorithm ?? (SR1)	1.53	4.39×10^{-9}	44	29	gradient

On MGH10 and MGH16, neither model reaches the gradient criterion, both stall under the step and function criteria, respectively, at essentially the same objective value, although Algorithm ?? with PSB reaches a smaller gradient norm on MGH10. For this reason, PSB is the best choice between the two approaches when weighing cost against benefit.

Table 3 repeats the comparison, but now running Algorithm ?? in full with PSB model on the same 34 problems. Newton/BFGS/BOBYQA solve only the small quartic model at each outer iteration, exactly as Algorithm ?? uses them, instead of standing in as the whole optimizer. "Newton" here is the exact Newton step on the quartic subproblem, computed via the Ipopt interior-point solver.

Function and gradient evaluations here count only the calls to the real residual and Jacobian made by Algorithm ??'s outer loop. The internal solver's own work on the quartic surrogate is tracked separately, since evaluating the true function and its derivative is considerably more expensive than evaluating the model. Because we are solving a quartic (rather than a quadratic) problem, we use a multistart method to search for a global optimum. At each outer iteration k, 100 random points are drawn from a ball $B(0, \max\{1, d_{k-1}\})$. Using the default multistart setting for every method would be impractical across all 34 problems in this table.

Table 3: Algorithm ?? (PSB), varying only `model_solver`, on the 34 MGH problems.

Method	f	Converged (gradient)	Iterations	Func. evals	Grad. evals
Algorithm ?? (PSB, Newton)	$1.29 \times 10^{3} \pm 7.36 \times 10^{3}$	31/34	15.12 ± 21.90	30.21 ± 49.59	16.12 ± 21.90
Algorithm ?? (PSB, BFGS)	$1.29 \times 10^{3} \pm 7.36 \times 10^{3}$	31/34	15.26 ± 21.99	30.26 ± 48.63	16.26 ± 21.99
Algorithm ?? (PSB, BOBYQA)	$1.29 \times 10^{3} \pm 7.36 \times 10^{3}$	30/34	39.74 ± 108.86	57.68 ± 127.48	40.74 ± 108.86

Newton and BFGS are essentially indistinguishable (31/34, within 1% of each other on every column), and even BOBYQA, which needed an order of magnitude more evaluations and converged on only 10/34 when applied directly to the raw residual, falls behind by only one problem (30/34) and roughly 2.6× more outer iterations on average. This distinction matters because evaluating the derivative can sometimes cost more than 2.6× what evaluating the objective function costs. Since Newton achieves the best performance among the three, we adopt it as the model solver used throughout the rest of this work.

2.2 sum of square problem BFGS configuration

Now we will consider a diferent configuration thant the avarege to BFGS method. We will make it, because the defalt linesearch, generally Hanger Zhang linesearch, evaluate the derivativo of function many times. For this reason, now we will test diferents classical linesearchs for BFGS method. The Table 4 checks some linesearchs for `Optim.BFGS` on the same 34 MGH problems with four line searches. Hager and Zhang (`Optim.jl`'s default), a minimalist backtracking with Armijo condition, and BackTracking with quadratic (order 2) and cubic (order 3) interpolation. "Converged (gradient)" counts how many of the 34 problems stopped under the gradient criterion; the remaining columns are mean ± standard deviation over all 34 problems (including the ones that stopped under a different criterion).

Table 4: Comparison of line searches for Optim.jl's BFGS on the 34 MGH problems.

Method	f	Converged (gradient)	Iterations	Func. evals	Grad. evals
Hager-Zhang	$1.29 \times 10^3 \pm 7.36 \times 10^3$	33/34	66.91 ± 120.92	107.82 ± 206.62	107.82 ± 206.62
Backtracking	$2.49 \times 10^7 \pm 1.45 \times 10^8$	30/34	51.56 ± 93.74	89.62 ± 131.90	52.47 ± 93.75
Backtracking (order 2)	$1.29 \times 10^3 \pm 7.36 \times 10^3$	32/34	61.59 ± 99.26	86.74 ± 134.56	62.56 ± 99.17
Backtracking (order 3)	$1.29 \times 10^3 \pm 7.36 \times 10^3$	31/34	56.91 ± 89.47	84.91 ± 132.01	57.85 ± 89.33

Hager-Zhang converges by gradient on the most problems (33/34) but at the highest average cost. the backtracking is a cheap linesearch but the value of f and congence per problem on average isn't good (30/34). The backtracking order 2 and 3 have a near results. Inded of Backtracking order 3 has a little less evaluations, the Backtracking order 2 found 32/34 gradients convegence. For this reason, wee will chose Backtracking order 2 to compare with our approach.

2.2.1 Algorithm ?? against BFGS with backtracking

Algorithm **??** (PSB) uses persistent decaying σ, a full Hessian reset on a bad ratio, and $-\nabla f$+Armijo at $k = 0$ and as a safeguard. The following table compares it against BFGS with backtracking line search on the 34 MGH least-squares problems.

Table 5: Algorithm **??** (PSB) vs. BFGS (backtracking): per-problem comparison on the MGH problems.

Problem	Method	f	$\|\nabla f\|$	Func. evals	Grad. evals	Criterion
MGH01	Algorithm **??** (PSB)	2.42×10^{-22}	9.82×10^{-12}	12	8	gradient
MGH01	BFGS (backtracking)	1.98×10^{-22}	3.69×10^{-10}	53	40	gradient
MGH02	Algorithm **??** (PSB)	5.10×10^{-18}	3.84×10^{-9}	10	7	gradient
MGH02	BFGS (backtracking)	1.11×10^{-26}	2.20×10^{-12}	15	11	gradient
MGH03	Algorithm **??** (PSB)	1.58×10^{-11}	2.57×10^{-9}	31	20	gradient
MGH03	BFGS (backtracking)	5.05×10^{-30}	2.83×10^{-10}	233	174	gradient
MGH04	Algorithm **??** (PSB)	3.81×10^{-17}	8.73×10^{-9}	10	8	gradient
MGH04	BFGS (backtracking)	9.86×10^{-32}	4.44×10^{-10}	40	14	gradient
MGH05	Algorithm **??** (PSB)	1.31×10^{-16}	6.95×10^{-9}	10	8	gradient
MGH05	BFGS (backtracking)	2.50×10^{-22}	1.08×10^{-10}	22	19	gradient
MGH06	Algorithm **??** (PSB)	1.01×10^{3}	1.16×10^{-28}	11	2	gradient
MGH06	BFGS (backtracking)	1.01×10^{3}	1.16×10^{-28}	10	2	gradient
MGH07	Algorithm **??** (PSB)	1.49×10^{-17}	6.03×10^{-9}	17	10	gradient
MGH07	BFGS (backtracking)	5.24×10^{-19}	5.03×10^{-9}	41	30	gradient
MGH08	Algorithm **??** (PSB)	4.11×10^{-3}	4.90×10^{-10}	12	9	gradient
MGH08	BFGS (backtracking)	4.11×10^{-3}	7.39×10^{-10}	34	28	gradient
MGH09	Algorithm **??** (PSB)	5.64×10^{-9}	7.55×10^{-11}	9	7	gradient
MGH09	BFGS (backtracking)	5.64×10^{-9}	1.55×10^{-9}	6	5	gradient
MGH10	Algorithm **??** (PSB)	4.40×10^{1}	1.20×10^{-5}	276	122	stalled
MGH10	BFGS (backtracking)	4.40×10^{1}	2.28	552	386	step
MGH11	Algorithm **??** (PSB)	5.96×10^{-21}	3.25×10^{-13}	26	17	gradient
MGH11	BFGS (backtracking)	2.27×10^{-20}	7.17×10^{-10}	50	42	gradient
MGH12	Algorithm **??** (PSB)	3.80×10^{-18}	3.81×10^{-10}	14	11	gradient
MGH12	BFGS (backtracking)	1.58×10^{-21}	3.20×10^{-11}	53	41	gradient

continued on next page

Table 5: (continued)

Problem	Method	f	$\|\nabla f\|$	Func. evals	Grad. evals	Criterion
MGH13	Algorithm **??** (PSB)	5.16×10^{-12}	9.61×10^{-9}	12	9	gradient
MGH13	BFGS (backtracking)	1.57×10^{-14}	3.61×10^{-9}	55	45	gradient
MGH14	Algorithm **??** (PSB)	9.42×10^{-19}	8.23×10^{-10}	11	7	gradient
MGH14	BFGS (backtracking)	3.52×10^{-22}	5.46×10^{-10}	44	31	gradient
MGH15	Algorithm **??** (PSB)	1.54×10^{-4}	9.06×10^{-9}	38	22	gradient
MGH15	BFGS (backtracking)	1.54×10^{-4}	4.13×10^{-10}	39	33	gradient
MGH16	Algorithm **??** (PSB)	4.29×10^{4}	6.53×10^{-5}	91	37	stalled
MGH16	BFGS (backtracking)	4.29×10^{4}	2.83×10^{-5}	72	26	function
MGH17	Algorithm **??** (PSB)	2.73×10^{-5}	8.38×10^{-9}	24	12	gradient
MGH17	BFGS (backtracking)	2.73×10^{-5}	8.23×10^{-9}	90	59	gradient
MGH18	Algorithm **??** (PSB)	1.00×10^{-18}	1.65×10^{-11}	58	34	gradient
MGH18	BFGS (backtracking)	2.83×10^{-3}	1.28×10^{-8}	44	41	gradient
MGH19	Algorithm **??** (PSB)	4.38×10^{-2}	9.20×10^{-9}	115	59	gradient
MGH19	BFGS (backtracking)	1.37×10^{-1}	8.29×10^{-10}	147	113	gradient
MGH20	Algorithm **??** (PSB)	1.14×10^{-3}	1.84×10^{-9}	13	10	gradient
MGH20	BFGS (backtracking)	1.14×10^{-3}	7.07×10^{-9}	49	39	gradient
MGH21	Algorithm **??** (PSB)	2.42×10^{-21}	3.11×10^{-11}	12	8	gradient
MGH21	BFGS (backtracking)	4.79×10^{-18}	1.50×10^{-9}	228	150	gradient
MGH22	Algorithm **??** (PSB)	4.52×10^{-14}	2.80×10^{-10}	13	10	gradient
MGH22	BFGS (backtracking)	1.65×10^{-13}	1.56×10^{-8}	136	112	gradient
MGH23	Algorithm **??** (PSB)	1.12×10^{-5}	2.28×10^{-9}	13	10	gradient
MGH23	BFGS (backtracking)	1.12×10^{-5}	1.21×10^{-8}	87	68	gradient
MGH24	Algorithm **??** (PSB)	1.73×10^{-6}	1.74×10^{-9}	37	22	gradient
MGH24	BFGS (backtracking)	1.73×10^{-6}	6.29×10^{-9}	588	450	gradient
MGH25	Algorithm **??** (PSB)	7.40×10^{-32}	3.85×10^{-16}	13	6	gradient
MGH25	BFGS (backtracking)	6.84×10^{-26}	7.27×10^{-12}	23	16	gradient
MGH26	Algorithm **??** (PSB)	1.40×10^{-5}	3.00×10^{-9}	27	16	gradient
MGH26	BFGS (backtracking)	1.40×10^{-5}	1.11×10^{-8}	36	34	gradient
MGH27	Algorithm **??** (PSB)	2.24×10^{-16}	1.94×10^{-9}	11	8	gradient
MGH27	BFGS (backtracking)	5.73×10^{-20}	3.69×10^{-9}	20	17	gradient
MGH28	Algorithm **??** (PSB)	2.22×10^{-15}	6.93×10^{-9}	10	8	gradient
MGH28	BFGS (backtracking)	2.53×10^{-16}	1.09×10^{-8}	26	18	gradient
MGH29	Algorithm **??** (PSB)	1.13×10^{-22}	1.91×10^{-11}	8	7	gradient
MGH29	BFGS (backtracking)	9.10×10^{-19}	1.56×10^{-9}	8	8	gradient
MGH30	Algorithm **??** (PSB)	7.64×10^{-27}	3.46×10^{-13}	10	7	gradient
MGH30	BFGS (backtracking)	1.97×10^{-19}	4.03×10^{-9}	46	23	gradient
MGH31	Algorithm **??** (PSB)	1.53	4.17×10^{-8}	49	18	stalled
MGH31	BFGS (backtracking)	1.34	4.48×10^{-9}	78	43	gradient
MGH32	Algorithm **??** (PSB)	5.00	3.85×10^{-16}	3	2	gradient
MGH32	BFGS (backtracking)	5.00	5.58×10^{-16}	2	2	gradient
MGH33	Algorithm **??** (PSB)	2.32	6.33×10^{-11}	12	5	gradient

continued on next page

Table 5: (continued)

Problem	Method	f	$\|\nabla f\|$	Func. evals	Grad. evals	Criterion
MGH33	BFGS (backtracking)	2.32	1.42×10^{-10}	13	5	gradient
MGH34	Algorithm **??** (PSB)	3.07	1.55×10^{-10}	9	2	gradient
MGH34	BFGS (backtracking)	3.07	2.13×10^{-10}	9	2	gradient

Across the 34 problems, the two methods usually agree closely on the final objective value, but differ sharply in cost. Algorithm **??** (PSB) needs far fewer function and gradient evaluations than BFGS on most problems (see Table 5). MGH10 shows how hard some of these problems are. Neither method certifies gradient convergence there, and both land on essentially the same objective value ($f \approx 43.97$), yet Algorithm **??** still reaches a far smaller gradient norm than BFGS ($\|\nabla f\| \approx 1.2 \times 10^{-5}$ against ≈ 2.28), using half the function evaluations and under a third of the gradient evaluations. MGH18 shows the opposite kind of gap. Although both methods stop under the gradient criterion, Algorithm **??** reaches essentially zero ($f \approx 1.0 \times 10^{-18}$) while BFGS settles for a distinctly larger value ($f \approx 2.8 \times 10^{-3}$). A small gradient norm does not always mean the same stationary point.

The boxplot below compares the two approaches directly and uses the Mann-Whitney U test to check whether the difference between them is statistically significant.

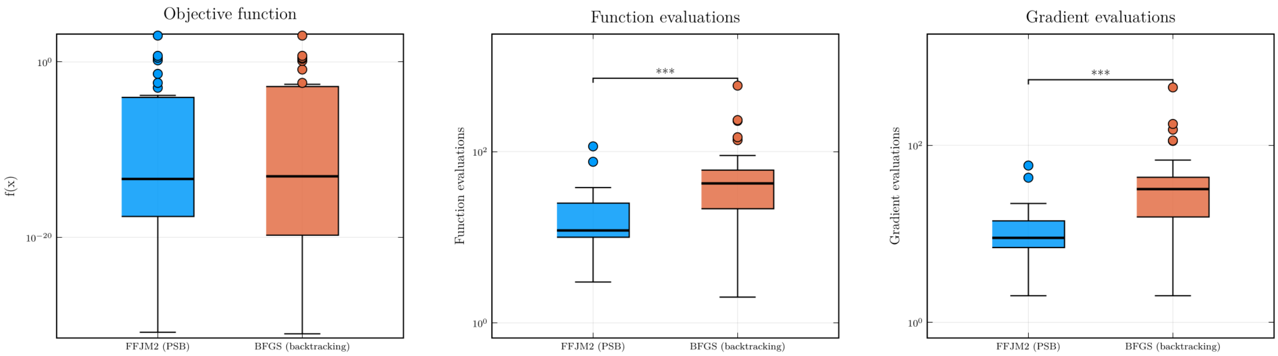

Figure 1: Comparison of Algorithm **??** (PSB) vs. BFGS (backtracking) on the MGH problems, restricted to runs where both methods converged by gradient norm. Final f value, function evaluations, and gradient evaluations. Stars indicate significance under the Mann-Whitney U test [6] ($p < 0.05$: *; $p < 0.01$: **; $p < 0.001$: ***).

As Figure 1 shows, the objective function values are not very different between the two methods. The p-value confirms neither achieves a significantly better result. Function evaluations and gradient evaluations, however, are different, and the p-value confirms this difference is significant. Algorithm **??** therefore performs better than BFGS in this respect, reaching a comparable solution at a much lower cost.

3 Coefficient estimation problem

The Saint-Venant equations [2, 5, 10, 11] describe the flow of fluids in natural channels. The state variables that describe a natural channel vary with respect to $x \in [x_{min}, x_{max}]$ (position, usually measured in meters) and $t \in [t_{min}, t_{max}]$ (time, usually measured in seconds). We define:

- $z(x, t)$: surface elevation
- $h(x, t) = z(x, t) - z_b(x)$: river depth at (x, t)
- $A(x, t)$: cross-sectional wetted area
- $Q(x, t)$: flow rate

- $P(x,t)$: wetted perimeter
- $R(x,t) = A(x,t)/P(x,t)$: hydraulic radius
- g: acceleration of gravity (approximately $9.81m/s^2$)

The Saint-Venant equations consist of two main components: Equation (4) describes mass conservation and equation (5) represents conservation of the linear momentum.

$$\frac{\partial A}{\partial t} + \frac{\partial Q}{\partial x} = 0 \tag{4}$$

and

$$\frac{\partial Q}{\partial t} + \frac{\partial}{\partial x}\left(\frac{Q^2}{A}\right) + gA\frac{\partial z}{\partial x} + \frac{n_g^2 Q|Q|}{AR^{4/3}} = 0. \tag{5}$$

The Manning (also called Yen-Manning [1]) coefficient n_g represents roughness [1]. Typically, given the inlet discharge $Q(x_{min}, t)$ for $t \in [t_{min}, t_{max}]$, initial conditions $A(x, t_{min})$ and $Q(x, t_{min})$, and the coefficient n_g, numerical methods compute the values of $Q(x,t)$ and $A(x,t)$ at a set of discrete "grid points" (x_ν, t_ν). This PDE system is solved using an explicit finite difference method with artificial diffusion of 0.99, following [5, 10]. Unfortunately, the Yen-Manning coefficient n_g is often unknown for natural channels and can vary with position x and even time t. For more details, see [1].

Following [4], this research assumes that the Manning coefficient n_g varies with position x, and that a set of elevation observations $z_{obs}(x_\nu, t_\nu)$ are available at n_{obs} distinct locations and times, indexed by $\nu = 1, \ldots, n_{obs}$. Consequently, the spatial distribution of $n_g(x)$, discretized over a suitable grid, becomes an unknown component of our problem. Furthermore, to ensure physically plausible results, minimal variation will be required for a function $n_g(x)$ that minimizes spatial oscillations while fitting the solution of the Saint-Venant equations to the observations.

Consider a grid with n_{grid} points $x_{min} \leq x_1 < \cdots < x_{n_{grid}} \leq x_{max}$. Collect the n_{grid} values $n_g(x_1), \ldots, n_g(x_{n_{grid}})$ into a parameter vector $\theta \in I\!R^{n_{grid}}$, with $\theta_i := n_g(x_i)$ for $i = 1, \ldots, n_{grid}$, kept notationally distinct from the spatial coordinate x. In [4], this calibration problem is posed as a constrained optimization problem, minimizing either the sum of squared deviations of θ from a reference value $\bar{n}$ or the sum of absolute deviations, subject to equality constraints requiring the simulated elevations to match the n_{obs} observations and a box constraint $0 \leq \theta_i \leq n_g^{\max}$; it is solved there by a Safeguarded Augmented Lagrangian method (Algorithm 2.1 of [4]), with BOBYQA or PRIMA handling the subproblems.

The two applications differ mainly in how the underlying PDE is solved. In this work we rely on a more stable PDE solver, which lets us compute the derivative of the objective with respect to θ accurately; this is precisely what enables the gradient-based comparison carried out below, namely BFGS and Algorithm **??** against the derivative-free BOBYQA. Figure 2 contrasts the previous and the current solver on the same problem.

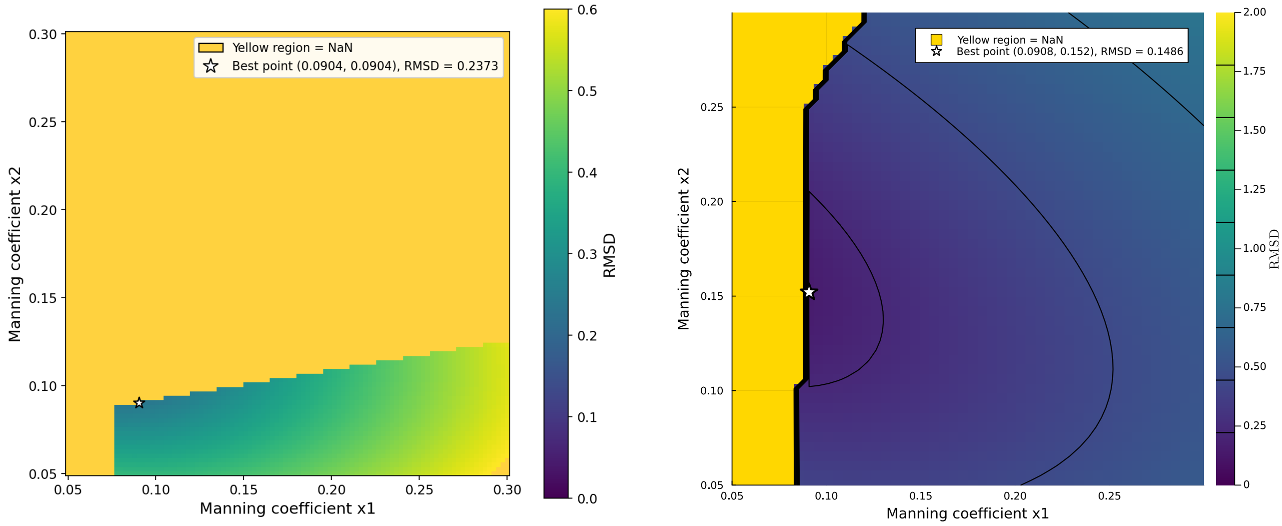

Figure 2: Comparison between the previous (left) and the current (right) PDE solver.

For the comparison in this section, we adapt this into the unconstrained nonlinear least-squares form used throughout this report, minimizing the elevation mismatch directly instead of enforcing it as an equality constraint. For $i = 1, \dots, m$ (with $m = n_{obs}$), let $z_i(\theta)$ denote the surface elevation the Saint-Venant solution predicts, for parameter vector θ, at the i-th observation's location and time (x_i, t_i), and let $z_{obs,i} = z_{obs}(x_i, t_i)$ be the corresponding observation. Define the residual components $f_i(\theta) = z_i(\theta) - z_{obs,i}$, collect them into $F(\theta) = (f_1(\theta), \dots, f_m(\theta))^T$, and minimize

$$f(\theta) = \frac{1}{2}\|F(\theta)\|_2^2 = \frac{1}{2}\sum_{i=1}^{m} f_i(\theta)^2.$$

Calibrating n_g against the observations is thus the nonlinear least-squares problem of finding θ that minimizes $f(\theta)$ (ideally $F(\theta) = 0$, though the real data need not be exactly consistent with the model).

3.1 Why consider this scheme

Solving a coefficient-estimation problem for a PDE is expensive. Evaluating the objective function requires solving the PDE once, and differentiating that objective requires differentiating the PDE solver itself. In [4] we solved this problem without derivatives, but the resulting computational cost was prohibitively high. In this work we instead use derivatives obtained by automatic differentiation.

This approach computes an exact derivative of the PDE solver, rather than a finite-difference approximation, via forward-mode automatic differentiation, using Julia's `ForwardDiff.jl`. We chose forward mode over reverse (backward) mode because reverse-mode differentiation must first record every elementary operation performed by the solver onto a computational tape before it can evaluate the derivative, and since the PDE solver performs a very large number of operations, this tape would require a prohibitive amount of memory, on the order of 3 gigabytes for a single derivative evaluation in our setting.

Using as much derivative information as possible is valuable in principle, but Figure 3 shows why this is impractical here. The cost of computing the exact Hessian by automatic differentiation grows far faster with the problem dimension than the cost of the gradient, so exact second-order information should be avoided, relying instead on first-order (gradient) information alone.

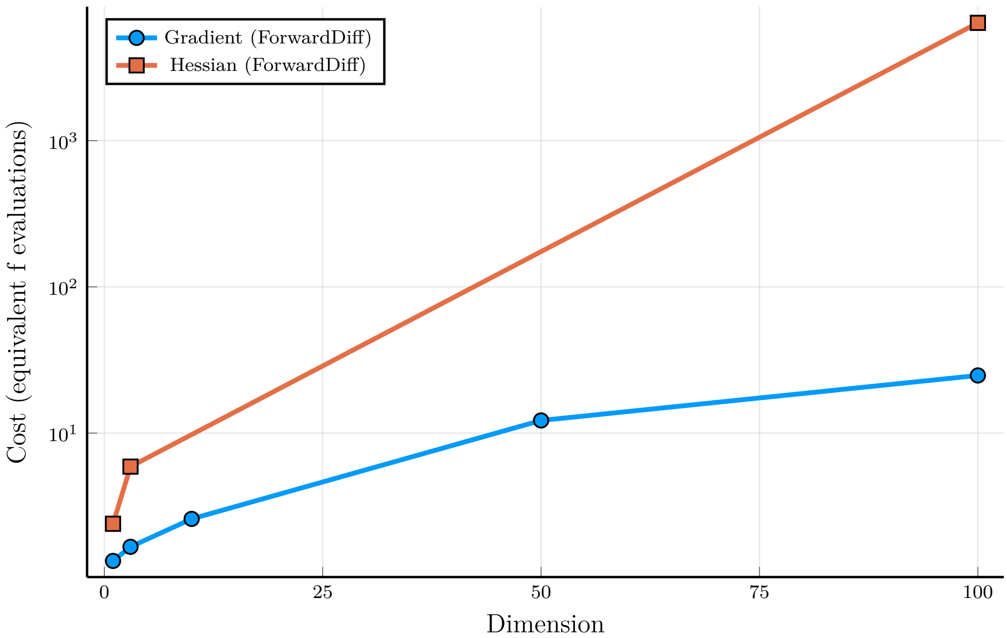

Figure 3: Cost, in equivalent evaluations of f, of computing the gradient and the Hessian of the PDE solver's objective by automatic differentiation, as a function of the problem dimension.

At dimension 100, Figure 3 shows that a single gradient evaluation already costs about as much as 25 evaluations of f, while a single Hessian evaluation costs more than 6400 evaluations of f.

3.2 Results in a coefficient estimation problem

To calibrate θ against this residual, four benchmark scenarios compare the same three solvers used throughout this report, namely BFGS (backtracking, order 2), BOBYQA, and Algorithm **??**, applied directly to $F(\theta)$ above where each solver stands in as the whole optimizer, rather than as Algorithm **??**'s internal subproblem solver. Two scenarios are pregenered experiments, where the observations $z_{obs,i}$ are themselves generated by the model at a known $\theta^\star$, so the true minimizer is known exactly; the other two use the real observed data, where the true minimizer (if it exists) is unknown. BFGS and BOBYQA minimize the full sum of squares $\frac{1}{2}\|F(\theta)\|_2^2 + \text{penalty}(\theta)$, where penalty is an additive box term (bounds $[0, 0.5]^{n_{grid}}$, weight 10^6) that evaluates to exactly 0 whenever every coordinate of θ is feasible, as is the case at every minimizer reported below; Algorithm **??** minimizes $f(\theta) = \frac{1}{2}\|F(\theta)\|_2^2$ directly. We consider a box ther for the BFGS and BOBYQA methods to help the convergence of solver.

Table 6: Twin experiment, $n_{grid} = 2$: $\theta^\star = (0.20,\ 0.15)$, $\theta^0 = (0.09,\ 0.09)$.

Method	RMSD	f	$\|\nabla f\|$	Func. evals	Grad. evals	Time (s)	Status
BFGS (backtracking)	6.95×10^{-9}	1.63×10^{-14}	1.19×10^{-5}	17	9	416.1	`GradientNorm`
BOBYQA	1.56×10^{-9}	8.20×10^{-16}	2.91×10^{-6}	43	0	456.2	`XTOL_REACHED`
Algorithm **??**	3.04×10^{-15}	1.57×10^{-27}	1.36×10^{-13}	11	6	200.5	`gradient_converged`

All three methods converge to essentially the exact reference $\theta^\star = (0.20, 0.15)$; Algorithm **??** is the cheapest of the three by every cost measure (fewest evaluations, least time) despite also reaching the tightest gradient norm and RMSD.

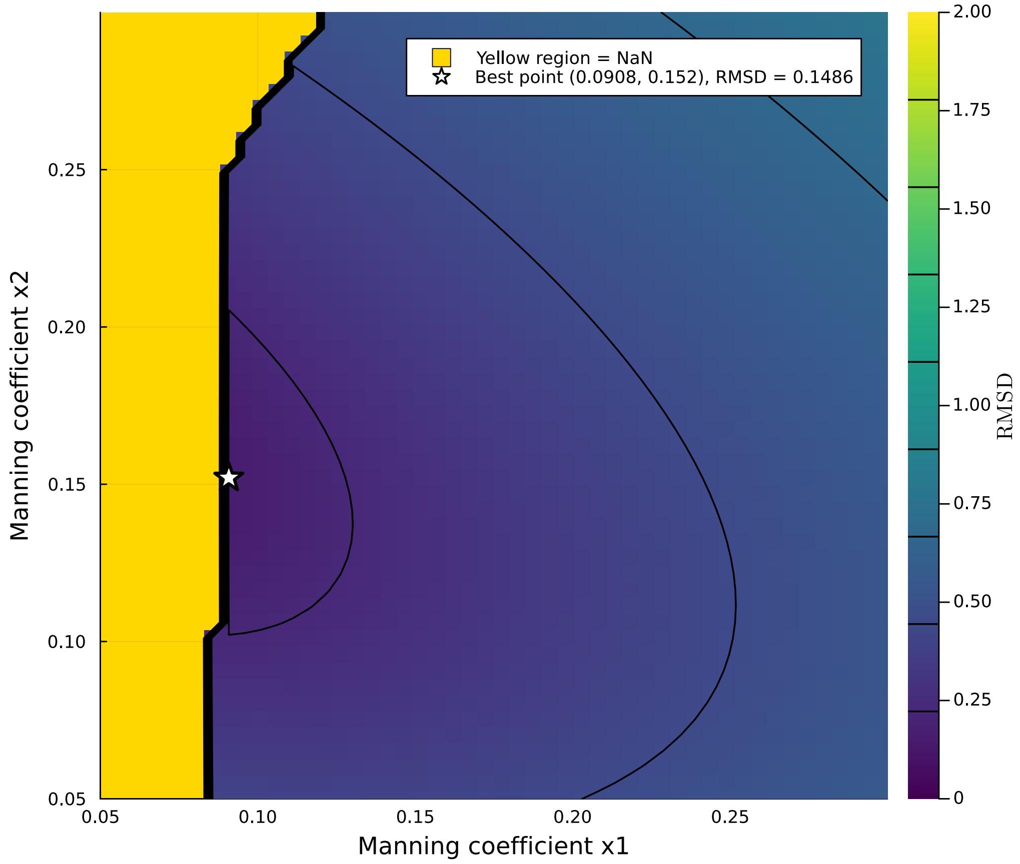

Figure 4: Cost, in equivalent evaluations of f, of computing the gradient and the Hessian of the PDE solver's objective by automatic differentiation, as a function of the problem dimension.

Table 7: Twin experiment, $n_{grid} = 10$: decreasing profile $\theta^\star$ from 0.12 to 0.07, $\theta^0 = (0.09, \dots, 0.09)$. (Increasing profiles over the same range were tested manually and diverge before $t_{\max} = 31$; only decreasing profiles are stable.)

Method	RMSD	f	$\|\nabla f\|$	Func. evals	Grad. evals	Time (s)	Status
BFGS (backtracking)	1.74×10^{-6}	1.03×10^{-9}	1.32×10^{-4}	205	168	9959.2	`GradientNorm`
BOBYQA	6.80×10^{-5}	1.56×10^{-6}	3.27×10^{-4}	1000	0	12037.3	`MAXEVAL_REACHED`
Algorithm ??	5.02×10^{-13}	4.26×10^{-23}	1.89×10^{-11}	29	17	942.7	`gradient_converged`

At $n_{grid} = 10$ the gap between Algorithm ?? and the other two widens sharply. It reaches a gradient norm roughly 7 orders of magnitude smaller than BFGS or BOBYQA, using about 7–35× fewer function evaluations and 10–13× less wall-clock time. BOBYQA exhausts its 1000-evaluation budget without reaching the gradient criterion (`MAXEVAL_REACHED`, `converged=false`).

Table 8: Real data, $n_{grid} = 3$: $\theta^0 = (0.09, 0.09, 0.09)$.

Method	RMSD	f	$\|\nabla f\|$	Func. evals	Grad. evals	Time (s)	Status
BFGS (backtracking)	5.46×10^{-2}	1.0085	3.85×10^{-6}	25	15	738.5	`GradientNorm`
BOBYQA	5.46×10^{-2}	1.0085	1.06×10^{-4}	116	0	1432.3	`XTOL_REACHED`
Algorithm ??	5.46×10^{-2}	0.5043	6.41×10^{-9}	26	12	553.6	`gradient_converged`

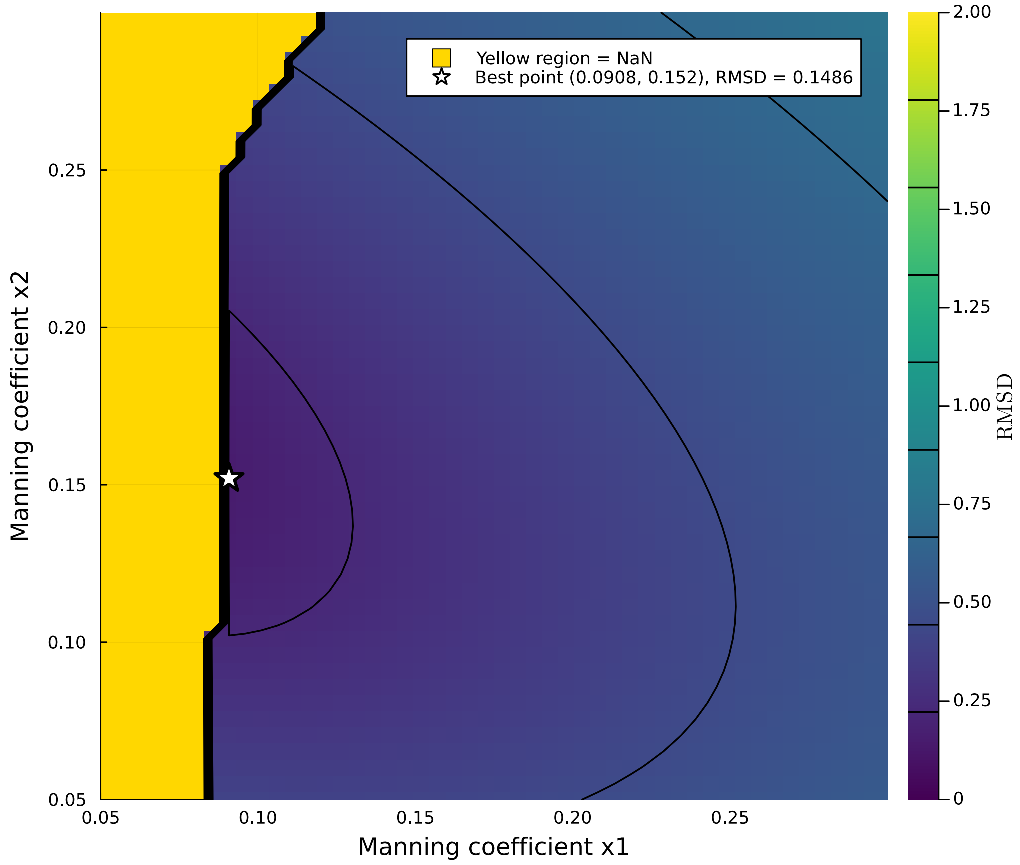

Figure 5: Cost, in equivalent evaluations of f, of computing the gradient and the Hessian of the PDE solver's objective by automatic differentiation, as a function of the problem dimension.

On real data, all three methods agree closely on the minimizer itself. RMSD matches to three significant figures across the board ($\approx 5.46 \times 10^{-2}$), and f matches once Algorithm **??**'s factor of 2 (discussed above) is accounted for. Algorithm **??** again reaches by far the smallest gradient norm, roughly three orders of magnitude below BFGS, at an evaluation count and wall-clock time comparable to BFGS.

Table 9: Real data, $n_{grid} = 10$: $\theta^0 = (0.09, \ldots, 0.09)$.

Method	RMSD	f	$\|\nabla f\|$	Func. evals	Grad. evals	Time (s)	Status
BFGS (backtracking)	4.81×10^{-2}	0.7822	1.19×10^{6}	171	35	3691.9	`alpha_too_small`
BOBYQA	4.64×10^{-2}	0.7285	1.11	840	0	10388.5	`SUCCESS`
Algorithm **??**	4.89×10^{-2}	0.4049	1.29	1008	501	31705.6	`maximum_iterations`

This is the hardest of the four scenarios, and the only one where none of the three methods reaches a stationary point. BFGS's backtracking line search stalls (`alpha_too_small`, `converged=false`) with a gradient norm still at $\sim 1.19 \times 10^{6}$, many orders of magnitude away from a stationary point, despite needing fewer evaluations and less time than BOBYQA or Algorithm **??**. BOBYQA reports `SUCCESS` (its own termination code, not a gradient-based one — it never evaluates ∇f) and reaches the lowest RMSD (4.64×10^{-2}), but needs 840 evaluations and $\approx$ 2.9h of wall-clock time to get there. Algorithm **??** exhausts its iteration budget (`maximum_iterations`) after 1008 residual and 501 Jacobian evaluations ($\approx$ 8.8h): it reaches a gradient norm of ≈ 1.29, between BFGS's stall point and a stationary point, at the highest evaluation count and wall-clock cost of the three. Unlike the easier scenarios above, more evaluations do not translate into a better outcome here — all three land on different minimizers with comparable RMSD (4.6–4.9×10^{-2}), and none certifies first-order optimality; MADS is omitted from this table — its run at this dimension did not finish cleanly (the saved output was incomplete) and is left for a future update.

References

[1] E. G. Birgin, M. R. Correa, V. A. González-López, J. M. Martínez, and D. S. Rodrigues, *Randomly supported variation of deterministic models and its application to one-dimensional shallow water flows*, J. Hydraulic Eng., 150 (2024), 04024026.

[2] O. Castro-Orgaz and W. H. Hager, *Shallow Water Hydraulics*, Springer, 2019.

[3] J. E. Dennis, Jr. and H. Wolkowicz, *Sizing and least-change secant methods*, SIAM J. Numer. Anal., 30 (1993), pp. 1291–1314.

[4] F. A. Fortunato and J. M. Martínez, *KKT-free Constrained Optimization and Application to River-Flow Modeling*, IMECC, University of Campinas, preprint, 2025.

[5] R. J. LeVeque, *Numerical Methods for Conservation Laws*, Lectures in Mathematics, ETH Zürich, Birkhäuser, 1992.

[6] H. B. Mann and D. R. Whitney, *On a test of whether one of two random variables is stochastically larger than the other*, Ann. Math. Statist., 18 (1947), pp. 50–60.

[7] P. K. Mogensen and A. N. Riseth, *Optim: A mathematical optimization package for Julia*, J. Open Source Software, 3 (2018), p. 615.

[8] J. J. Moré, B. S. Garbow, and K. E. Hillstrom, *Testing unconstrained optimization software*, ACM Trans. Math. Software, 7 (1981), pp. 17–41.

[9] M. J. D. Powell, *A new algorithm for unconstrained optimization*, in Nonlinear Programming, J. B. Rosen, O. L. Mangasarian, and K. Ritter, eds., Academic Press, New York, 1970, pp. 31–65.

[10] R. M. Porto, *Hidráulica Básica*, EESC-USP, São Paulo, 2000.

[11] A. J. C. Saint-Venant, *Théorie du mouvement non-permanent des eaux, avec application aux crues des rivière et à l'introduction des marées dans leur lit*, Comptes Rendus des Séances de Académie des Sciences, 73 (1871), pp. 147–154.